

```
In [1]: import pandas as pd
```

```
In [2]: # TASK 1: Flipkart-style Product Sales
flipkart = pd.DataFrame({"ProductName": ["Laptop", "Mobile", "Headphones", "Smart w",
"Price": [55000, 25000, 2000, 3500, 1200], "Qty": [2, 3, 5, 2, 4]})
```

```
In [3]: flipkart["TotalValue"] = (flipkart["Price"] * flipkart["Qty"])

print("TASK 1 - Flipkart Product Sales")
print(flipkart)
```

TASK 1 - Flipkart Product Sales

	ProductName	Price	Qty	TotalValue
0	Laptop	55000	2	110000
1	Mobile	25000	3	75000
2	Headphones	2000	5	10000
3	Smart Watch	3500	2	7000
4	Keyboard	1200	4	4800

```
In [4]: # TASK 2: Food Items - 10% Discount using apply()
food = pd.DataFrame({"Item": ["Pizza", "Burger", "Pasta", "Sandwich", "Biryani"],
"Price": [300, 200, 250, 150, 350],
"Qty": [2, 3, 1, 2, 1]})
```

```
In [5]: food["DiscountedPrice"] = food["Price"].apply(lambda price: price * 0.90)

print("\nTASK 2 - Food Items with Discounted Price")
print(food)
```

TASK 2 - Food Items with Discounted Price

	Item	Price	Qty	DiscountedPrice
0	Pizza	300	2	270.0
1	Burger	200	3	180.0
2	Pasta	250	1	225.0
3	Sandwich	150	2	135.0
4	Biryani	350	1	315.0

```
In [6]: # TASK 3: Instagram - Format Likes and Comments
instagram = pd.DataFrame({"PostID": [101, 102, 103, 104], "Likes": [1500, 2500000]
```

```
In [7]: def format_number(number):
    if number >= 1000000:
        return f"{number / 1000000:.1f}M"
    elif number >= 1000:
        return f"{number / 1000:.1f}K"
    else:
        return str(number)
```

```
In [9]: instagram[["Likes", "Comments"]] = instagram[["Likes", "Comments"]].map(format_n

#in new installed Pandas Version applymap has been removed and replaced with map

print("\nTASK 3 - Instagram Formatted Likes and Comments")
print(instagram)
```

TASK 3 - Instagram Formatted Likes and Comments

	PostID	Likes	Comments
0	101	1.5K	250
1	102	2.5M	15.0K
2	103	850	1.2K
3	104	1.2M	250.0K

```
In [10]: # TASK 4: Zomato - Final Amount using apply()
zomato = pd.DataFrame({"Restaurant": ["Dominos", "McDonalds", "Pizza Hut", "Subway"],
"Price": [300, 200, 350, 180, 250],
"Qty": [2, 3, 1, 2, 2],
"DeliveryCharge": [40, 30, 50, 25, 40]})
```

```
In [11]: zomato["FinalAmount"] = zomato.apply(lambda row: (row["Price"] * row["Qty"]) + row["DeliveryCharge"], axis=1)
```

```
In [12]: print("\nTASK 4 - Zomato Final Amount")
print(zomato)
```

TASK 4 - Zomato Final Amount

	Restaurant	Price	Qty	DeliveryCharge	FinalAmount
0	Dominos	300	2	40	640
1	McDonalds	200	3	30	630
2	Pizza Hut	350	1	50	400
3	Subway	180	2	25	385
4	Biryani House	250	2	40	540

```
In [13]: # TASK 5: Myntra - GST Calculation
myntra = pd.DataFrame({"Product": ["T-Shirt", "Jeans", "Shoes", "Jacket", "Wach"], "Price": [800, 1500, 2500, 3000, 2000], "Qty": [2, 1, 1, 2, 1]})
```

```
In [14]: #AI genrated Code

myntra["GST"] = (myntra["Price"] * myntra["Qty"] * 0.05)

print("\nTASK 5 - Myntra Orders with GST")
print(myntra)
```

TASK 5 - Myntra Orders with GST

	Product	Price	Qty	GST
0	T-Shirt	800	2	80.0
1	Jeans	1500	1	75.0
2	Shoes	2500	1	125.0
3	Jacket	3000	2	300.0
4	Wach	2000	1	100.0

```
In [15]: print("\nWhat I Learned:")
print("1. I learned how to create new DataFrame columns using calculations and apply().")
print("2. I learned how lambda functions can simplify row-wise calculations such as discounts, final amounts, and GST.")
```

What I Learned:

1. I learned how to create new DataFrame columns using calculations and apply().
2. I learned how lambda functions can simplify row-wise calculations such as discounts, final amounts, and GST.

```
In [ ]:
```